

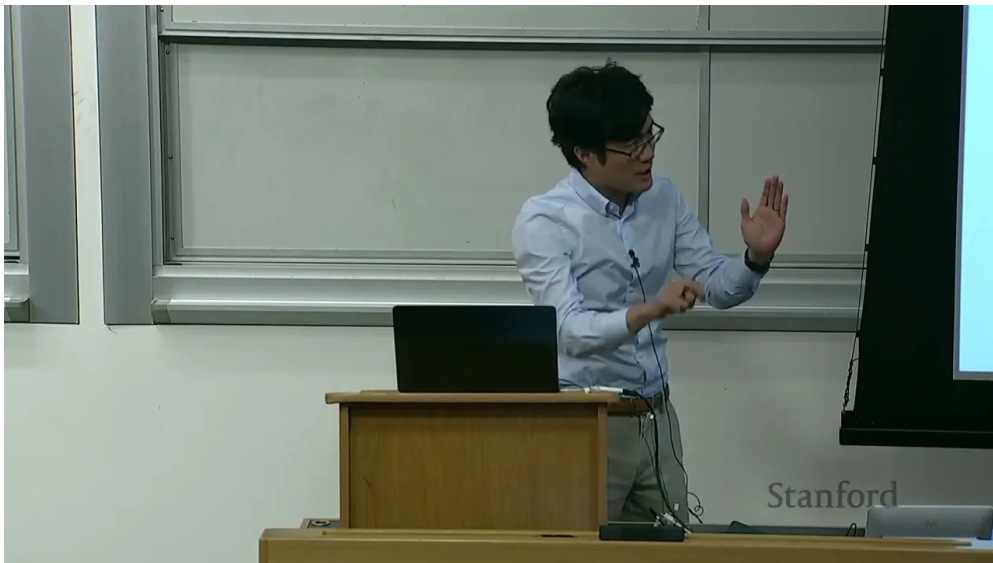
课程笔记

CS336 Lecture 11: Scaling Laws 2

Stanford CS336: Language Modeling from Scratch (Spring 2025)

讲师: Percy Liang, Tatsu Hashimoto 笔记: ZCode

2026-07-14



视频作者/频道: Stanford CS336

发布日期: 2025 Spring

视频时长: 78 分钟

视频链接: <https://www.youtube.com/watch?v=0SYuUqGBQxw>

Contents

1	导论：Scaling Laws 的第二幕	2
1.1	本节定位与回顾	2
1.2	为什么需要“更好的”Scaling Laws?	2
1.3	本节路线图	3
2	Better Scaling Laws: Chinchilla 方法论	4
2.1	参数化损失函数：从单变量到双变量	4
2.2	三步法：从损失函数到最优分配	5
2.3	参数化拟合的实验流程	6
2.4	IsoFLOP 曲线：可视化最优前沿	7
2.5	完整损失公式的拟合结果	9
2.6	学习率调度的影响	10
2.7	Compute-Optimal 前沿	11
2.8	Chinchilla 的三种独立方法	12
3	Scaling Laws 的真正含义	14
3.1	它们在统计什么?	14
3.2	数据质量改变 Scaling Law	15
3.3	下游任务 vs. 训练损失	15
3.4	Scaling Law 的适用边界	15
4	超参数缩放：一个被忽视的问题	16
4.1	问题陈述：从 N_{small} 到 N_{large}	16
4.2	宽度缩放：以宽度 m 为桥梁	17
4.3	实验观察：学习率应如何缩放	19
4.4	验证：从小模型迁移到大模型	20
5	Maximal Update Parametrization (μP)	21
5.1	动机：从经验观察到理论框架	21
5.2	激活值与梯度的尺度分析	22
5.3	μP 的完整推导	23
5.4	初始化稳定性	24
5.5	宽度缩放推导：输出层 vs. 隐藏层	25
5.6	学习率与宽度的最终关系	26
5.7	Batch size 的缩放	26
6	μP 的总结与实践意义	27
6.1	Summary: μP 提供了什么	27
6.2	μP 与 Chinchilla 的关系	28
6.3	实际模型如何偏离 Chinchilla	28
6.4	μP 的局限与开放问题	29
7	总结与延伸	29
7.1	本节核心论点	29
7.2	下节预告	30
7.3	配套阅读	30

1 导论：Scaling Laws 的第二幕

1.1 本节定位与回顾

这是 Scaling Laws 两连讲中的第二节。上一节 (Lecture 10) 我们建立了 Kaplan 式的经验性 Scaling Laws：损失随参数量 N 、数据量 D 和计算量 C 呈幂律下降，并推导了给定计算预算下的最优分配。本节将深入剖析 Scaling Laws 的工程实践细节，特别是 Chinchilla 方法的可复现性、含义，以及一个长期被忽视却至关重要的子问题——超参数迁移 (Hyperparameter Transfer)。



Figure 1: 开场回顾：上一节建立的 Kaplan 式经验 Scaling Law 的局限¹

1.2 为什么需要“更好的”Scaling Laws?

Kaplan 等人 2020 年的原始 Scaling Law 论文存在几个工程痛点，使其结论在实践中难以直接使用。讲师明确指出了三个关键问题：

Kaplan Scaling Law 的三个痛点

1. 学习率调度耦合：Kaplan 实验使用固定的 cosine schedule，每条训练曲线对应一个特定的 LR schedule。要估计 IsoFLOP 曲线上的每一个点，都得重新调 schedule——这在计算上极其昂贵。
2. α 与 β 难以分离：Kaplan 拟合的 $L(C) = a \cdot C^{-\alpha}$ 给出了“计算 $\rightarrow$ 损失”的整体幂律，但没有显式区分“参数不足”和“数据不足”两种不同的低效来源。
3. 最优 token/parameter 比例不明：Kaplan 得出“参数和 token 按相近速率增长”的结论，但这导致 GPT-3 等模型严重欠训练 (undertrained) ——参数虽多，但训练 token 远不够。

Chinchilla 论文 (Hoffmann et al., 2022) 的核心贡献，正是用一种更严谨、更可复现的实验设计，给出了完全不同的最优 token/parameter 比例：从 Kaplan 的约 1.7 tokens/param 修正到约 20 tokens/param。这一发现直接改变了后续所有大模型的训练策略。

¹ 视频画面时间区间：00:00:15 – 00:00:45。

核心结论预告

给定计算预算 C ，最优参数量 N^* 和最优 token 数 D^* 应满足：

$$N^* \propto C^{0.5}, \quad D^* \propto C^{0.5}$$

即两者按相同速率增长，且 $D^*/N^* \approx 20$ 。这意味着 Chinchilla 的最优策略是“中等大小但训练充分”，而非 Kaplan 的“大但欠训练”。

1.3 本节路线图

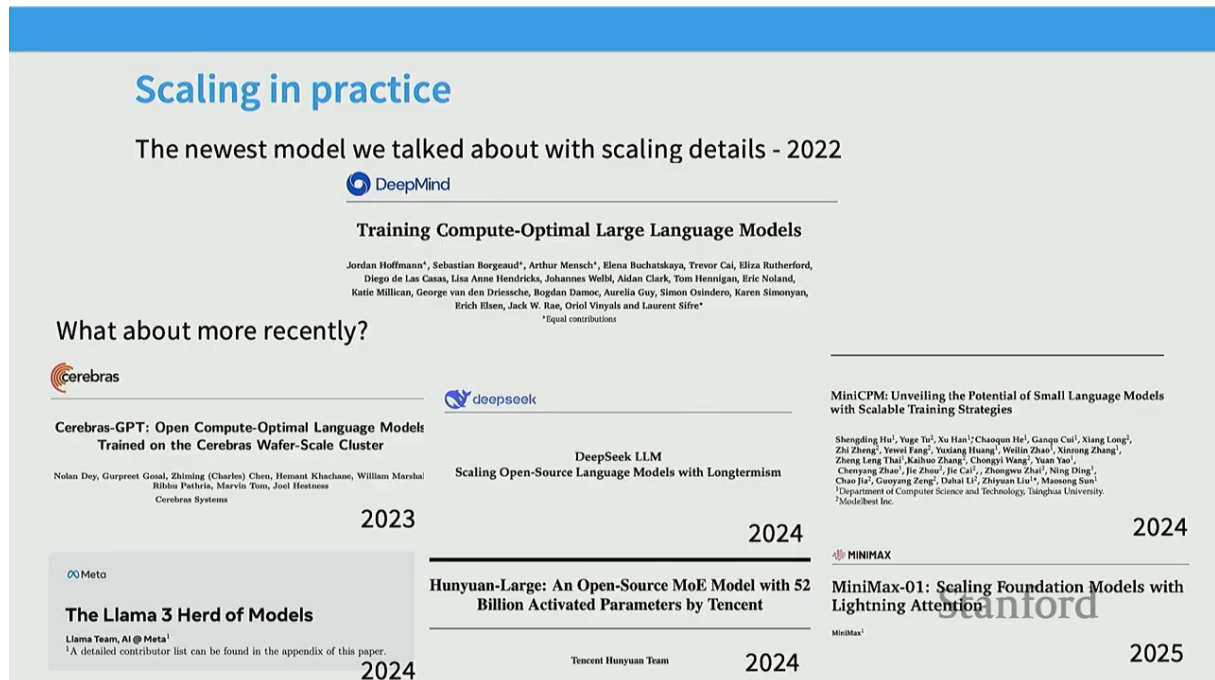


Figure 2: Better scaling laws: 本节将系统讲解 Chinchilla 方法论²

本节分为两大模块：

- 模块一（前半段）：深入 Chinchilla 的方法论——参数化拟合、IsoFLOP 曲线、三种独立估计方法、以及如何解读 Scaling Law 的“含义”。
- 模块二（后半段）：超参数迁移——当模型从 N_{small} 扩大到 N_{large} 时，学习率、batch size 等超参数应如何缩放？这是一个在 Kaplan/Chinchilla 框架之外，却同样决定成败的关键问题，最终引出 μP (Maximal Update Parametrization)。

²视频画面时间区间：00:02:00 – 00:02:30。

2 Better Scaling Laws: Chinchilla 方法论

2.1 参数化损失函数：从单变量到双变量

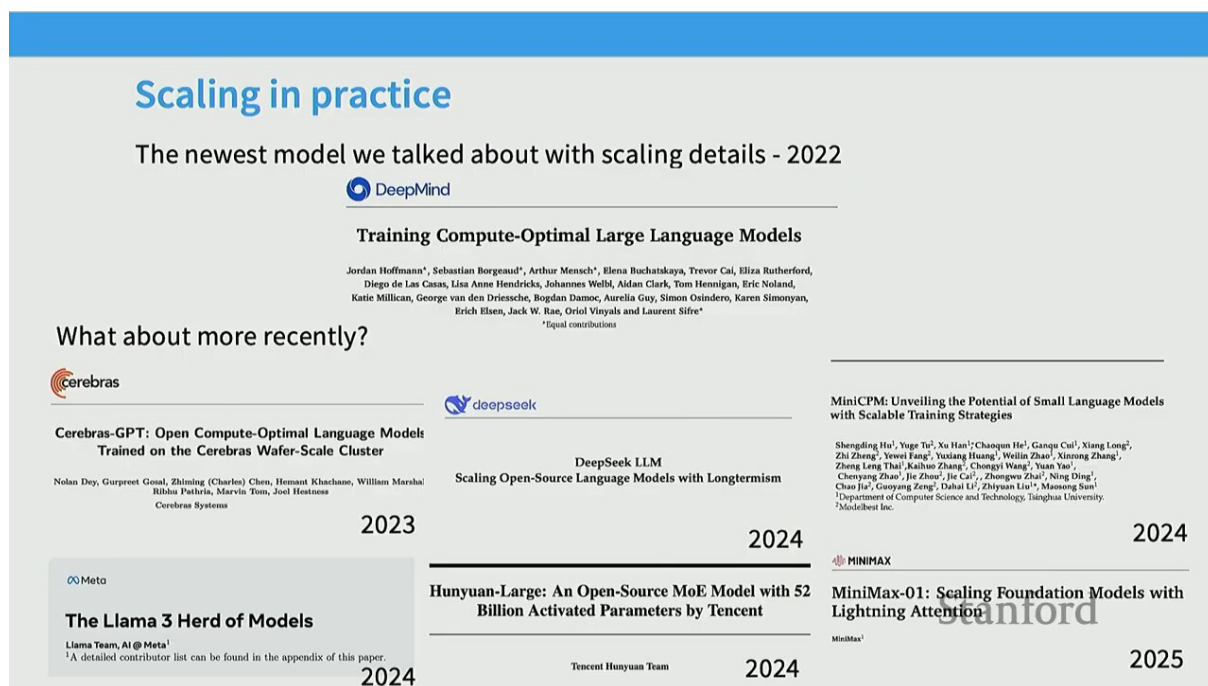


Figure 3: Better scaling laws: 双变量损失函数 $L(N, D)$ 的形式³

Chinchilla 的第一步，是把损失写成参数量 N 和数据量 D 的联合函数，而非 Kaplan 的“通过计算量 C 隐式关联”：

$$L(N, D) = E + \frac{A}{N^{\alpha_N}} + \frac{B}{D^{\alpha_D}}$$

- E ：不可约损失（irreducible loss），即数据集本身的信息熵下界，无论模型多大、训练多久都无法低于此值。对应自然语言的“真实困惑度”。
- A, α_N ：参数不足项，刻画“模型容量不够”导致的损失。
- B, α_D ：数据不足项，刻画“见过的 token 不够多”导致的损失。
- N ：模型参数量（不含 embedding 时不计词表参数）。
- D ：训练 token 数。

为何用加法而非乘法？

加性形式 $E + A/N^{\alpha_N} + B/D^{\alpha_D}$ 隐含假设：“参数不足”和“数据不足”是两个独立的低效来源。这并非严格成立——参数不足时，多给数据边际收益也会下降。但加性形式在实验中拟合得很好，且数学上易于求导优化，因此成为事实标准。

³视频画面时间区间：00:03:15 – 00:03:45。

2.2 三步法：从损失函数到最优分配

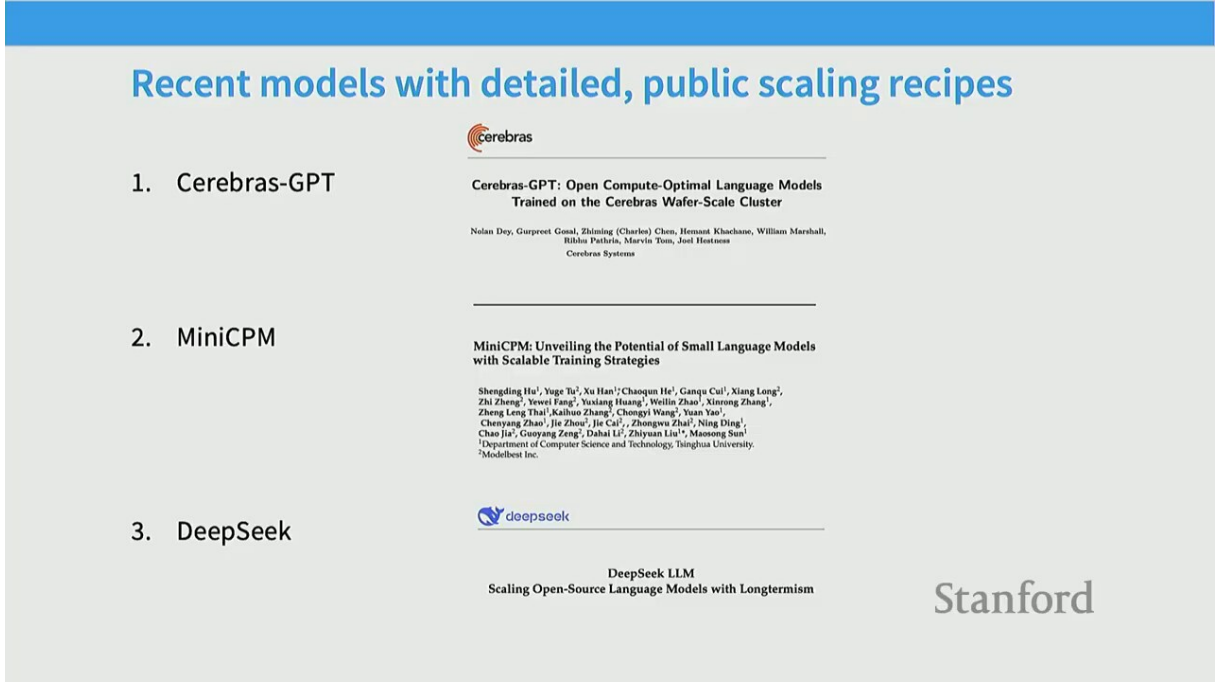


Figure 4: Chinchilla 三步法：参数化拟合 → 计算约束 → 求最优 N^*, D^* ⁴

给定损失函数 $L(N, D)$ 后，求“给定计算预算下的最优 (N^*, D^*) ”是一个标准的约束优化问题：

最优分配的数学表述

目标：最小化损失

$$\min_{N, D} L(N, D) = E + \frac{A}{N^{\alpha_N}} + \frac{B}{D^{\alpha_D}}$$

约束：计算量近似为 $C \approx 6ND$ （前向+反向，每参数每 token 共 6 FLOPs）

$$C = 6ND$$

用 Lagrange 乘子法：将 $D = C/(6N)$ 代入，对 N 求导令其为零：

$$\frac{\partial}{\partial N} \left[\frac{A}{N^{\alpha_N}} + \frac{B \cdot (6N)^{\alpha_D}}{C^{\alpha_D}} \right] = 0$$

解得最优 N^* 和 D^* 均与 $C^{0.5}$ 成正比：

$$N^*(C) = G_N \cdot C^a, \quad D^*(C) = G_D \cdot C^b$$

其中经验拟合给出 $a \approx b \approx 0.5$ ，且 $G_D/G_N \approx 20$ 。

这意味着最优策略是同时放大模型和数据，而非 Kaplan 建议的“主要放大模型”。两者的核心分歧可以用一个简单的比值刻画：

$$\frac{D^*}{N^*} \approx 20 \quad (\text{Chinchilla}) \quad \text{vs.} \quad \frac{D^*}{N^*} \approx 1.7 \quad (\text{Kaplan})$$

⁴视频画面时间区间：00:05:45 – 00:06:15。

GPT-3 欠训练了多少?

GPT-3 有 1750 亿参数，按 Kaplan 跑了约 3000 亿 token ($D/N \approx 1.7$)。但按 Chinchilla, 1750 亿参数的最优训练量应为 $1750 \times 20 = 3.5$ 万亿 token——GPT-3 的实际训练量不到最优值的 10%。DeepMind 据此用 700 亿参数的 Chinchilla 模型（训练 1.4 万亿 token）在多数 benchmark 上击败了 GPT-3，用更小的模型、更少总算力，实现了更好的效果。

2.3 参数化拟合的实验流程

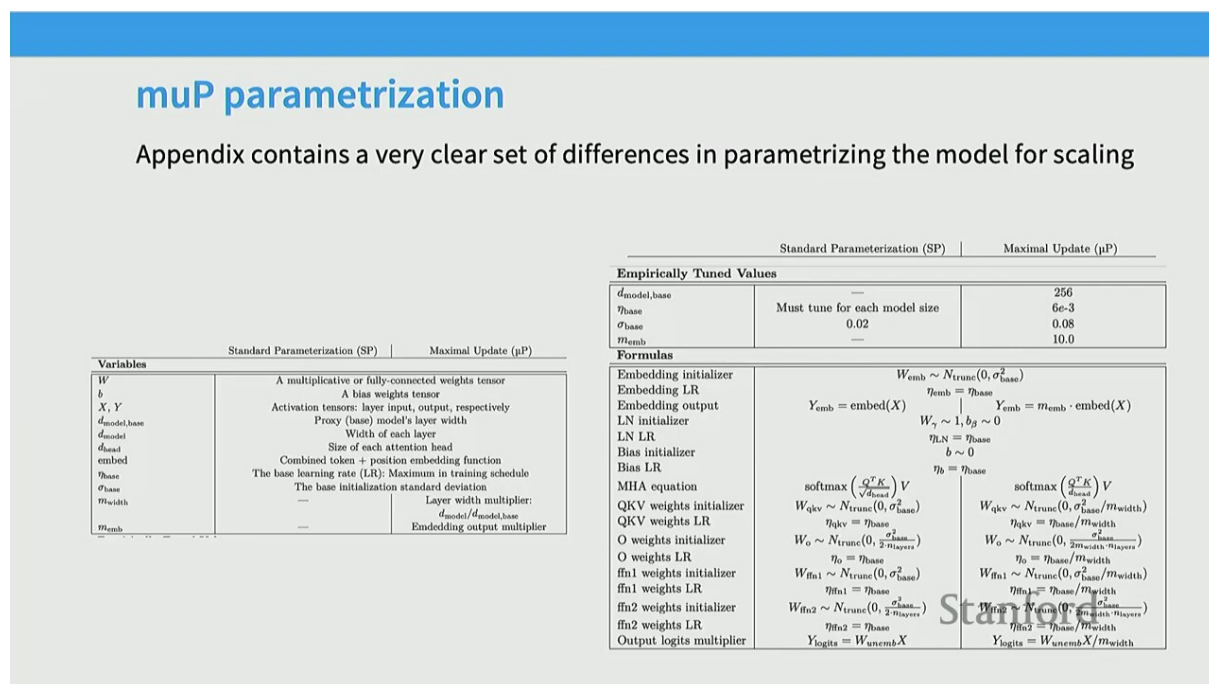


Figure 5: Better scaling laws: 参数化拟合的实验设计与数据采集⁵

要拟合 $L(N, D)$ 的六个未知数 ($E, A, B, \alpha_N, \alpha_D$) (E 通常单独估)，需要在多组 (N, D) 上训练模型，记录最终损失。讲师强调了几个工程细节：

1. 学习率调度必须可复现：每条训练曲线都应使用相同的 LR schedule 策略（如 cosine decay to 10% of peak），否则不同 (N, D) 的损失不可比。
2. 训练到收敛：每个 (N, D) 点都要训到 IsoFLOP 曲线的最低点附近，否则拟合的是“欠训练损失”而非“收敛损失”。
3. 参数量 N 的定义：Chinchilla 不计 embedding 参数，因为 embedding 只是查表，不参与核心的前向计算。这与 Kaplan 的统计口径不同，是两者结论差异的部分原因。

⁵视频画面时间区间：00:08:45 – 00:09:15。

实际拟合代码思路

```

1 import numpy as np
2 from scipy.optimize import curve_fit
3
4 def loss_fn(ND, E, A, B, aN, aD):
5     N, D = ND
6     return E + A / N**aN + B / D**aD
7
8 # ND: (N_i, D_i) ； L: ；
9 popt, _ = curve_fit(loss_fn, (N_arr, D_arr), L_arr,
10                      p0=[2.0, 100, 100, 0.3, 0.3],
11                      bounds=([1, 0, 0, 0, 0], [4, 1e6, 1e6, 1, 1]))
12 E, A, B, aN, aD = popt
13 # ； C, L(N, C/(6N))  N

```

Listing 1: Huber 损失下的 Scaling Law 拟合

2.4 IsoFLOP 曲线：可视化最优前沿

Setting the empirical values

muP is combined with aggressive scaling for hyperparameter optimization.

We tune μP hyperparameters on a 40M parameter proxy model, and $\mu Transfer$ those hyperparameters to our models with 111M-2.7B parameters. Figure 13 shows the results of a 200 sample random hyperparameter search on a 40M parameter proxy model ($d_{model} = d_{model,base} = 256$, $n_{layers} = 32$, $d_{head} = 128$) trained on 600M tokens with a batch size of 131k tokens. From this sweep we obtained the following tuned hyperparameters for our μP models: $\eta_{base} = 6e-3$, $\sigma_{base} = 0.08$, $m_{emb} = 10$. These hyperparameters also closely match those used by Yang et al. (2021).

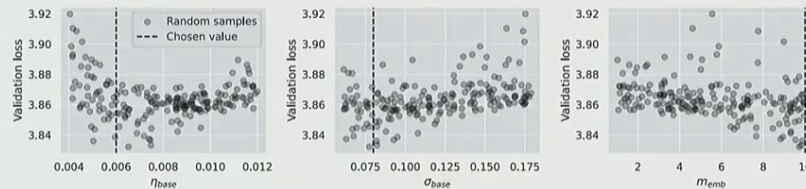


Figure 13: Results from random hyperparameter search on 40M parameter μP proxy model.

Generally stable hyperparameters.

Stanford

Figure 6: Better scaling laws: IsoFLOP 曲线——固定计算量下，损失随 N 的变化⁶

IsoFLOP 曲线是理解 Chinchilla 的关键可视化工具。固定计算量 C （即 $6ND = \text{const}$ ），扫描不同的 N （相应地 $D = C/(6N)$ ），画出损失随 N 的曲线。这条曲线呈 U 形：

- 左侧（ N 太小）：模型容量不足， A/N^{α_N} 项主导，损失高。
- 右侧（ N 太大）：参数多但 token 少， B/D^{α_D} 项主导（因为 $D = C/(6N)$ 随 N 增大而减小），损失回升。
- 谷底：两条衰退曲线的交点，即该 C 下的最优 N^* 。

⁶视频画面时间区间：00:10:30 – 00:11:00。

Techique 1: muP to stabilize scaling

Scale_emb = 12, scale_depth = 1.4, init_std = 0.1, lr = 0.01

Name	Specific Operation
Embedding Output Scaling	Multiply the output of the embedding by <i>scale_emb</i>
Residual Connection Scaling	Scale the increment at each residual connection in each layer by <i>scale_depth</i> / $\sqrt{\text{num_layers}}$
Initialization of Tensors	Set the initialization standard deviation of each two-dimensional tensor parameter to $\text{init_std} / \sqrt{d_m / d_{base}}$, and set other parameters' initialization to 0.1
Learning Rate Scaling of Tensors	Adjust the learning rate of each two-dimensional tensor parameter to $1 / (d_m / d_{base})$ times the learning rate of other parts (or the overall learning rate)
LM Head Scaling	Adjust the output logits to $1 / (d_m / d_{base})$ times the original value

Table 7: List of operations used when applying tensor program techniques.

c.f. CerebrasGPT – Scale_emb = 10, lr=6e-3, init_base = 0.08

Stanford

Figure 7: Better scaling laws: 等损失曲线族——每条曲线对应一个计算量⁷

将不同 C 的 IsoFLOP 谷底点 $(N^*(C), D^*(C))$ 连起来，就得到了计算最优前沿（compute-optimal frontier）。这条前沿在对数-对数坐标下是一条直线，斜率揭示了 N^* 和 D^* 各自随 C 的幂律指数。

前沿的几何意义

在 $(\log N, \log D)$ 平面上，等计算量线 $6ND = C$ 是一条斜率为 -1 的直线。最优前沿是这些直线与等损失椭圆的切点轨迹。前沿的斜率若为正且小于 -1 的绝对值（实际约为 -0.5 附近，即接近水平），意味着 N 和 D 应等比例增长。这正是 Chinchilla $a \approx b \approx 0.5$ 的几何直觉。

⁷视频画面时间区间：00:13:45 – 00:14:15。

2.5 完整损失公式的拟合结果

Optimal LR

According to muP – optimal learning rate should be (roughly) stable. Is it?

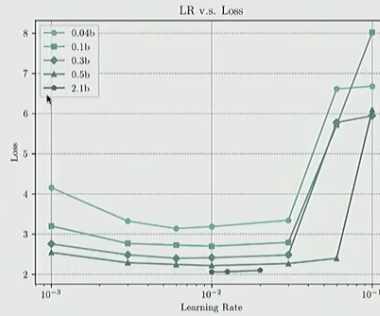


Figure 3: Loss vs Learning Rate. After applying for the Tensor Program, the learning rate shift becomes minimal.

Stanford

Figure 8: Better scaling laws: 完整参数化损失 $L(N, D) = E + A/N^\alpha + B/D^{\beta_8}$

Chinchilla 论文给出的拟合参数（数值因实现而略有差异，以下为典型量级）：

参数	典型值	含义
E	≈ 1.69 (nats)	不可约损失，对应数据集的信息熵下界
A	≈ 406	参数不足项的系数
B	≈ 410	数据不足项的系数
α_N	≈ 0.34	参数不足的幂律指数
α_D	≈ 0.28	数据不足的幂律指数

注意 $\alpha_N \approx \alpha_D$ （都在 0.3 附近），这是 N^* 和 D^* 按相同速率增长 ($C^{0.5}$) 的数学根源。如果 $\alpha_N > \alpha_D$ ，则参数不足的损失下降更快，最优策略会偏向“小模型多数据”；反之亦然。

α 值的敏感性分析

最优分配 $N^*(C)$ 的指数 a 不仅取决于 α_N, α_D ，还取决于两者之比。具体地：

$$a = \frac{\alpha_D}{\alpha_N + \alpha_D}$$

若 $\alpha_N = \alpha_D$ ，则 $a = 0.5$ 。Chinchilla 的拟合给出 $a \approx 0.46 - 0.5$ ，接近理论值。但 α 的估计本身有噪声—— ± 0.02 的波动就能让 a 在 0.45 - 0.55 间漂移。这也是为什么 Chinchilla 用三种独立方法交叉验证。

⁸ 视频画面时间区间：00:17:30 - 00:18:00。

2.6 学习率调度的影响

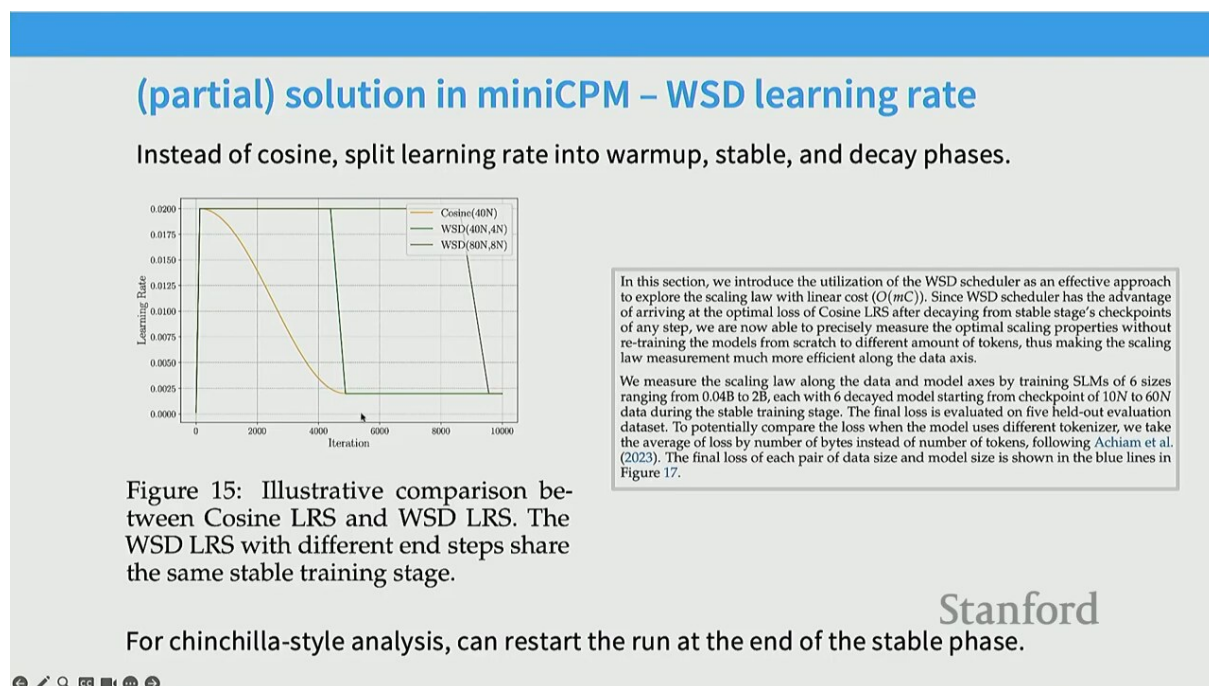


Figure 9: Better scaling laws: 学习率调度对损失曲线的影响⁹

讲师特别强调了学习率调度的复杂性。Kaplan 论文为每条曲线单独调 LR schedule，导致 IsoFLOP 曲线的不同点对应不同的训练动态，可比性存疑。Chinchilla 的改进是：

- 统一的 schedule 策略：所有训练运行使用相同的 cosine decay 形式（峰值 LR 因模型而异，但 decay 比例固定）。
- 训练到充分收敛：每个 (N, D) 点都训练到 IsoFLOP 曲线的最低点，而非固定 epoch 数。

“充分收敛”的陷阱

判断“充分收敛”本身需要经验。如果训练步数不够，拟合出的 α 会偏小（因为损失还在下降，没有触底）。讲师提到一个实用技巧：用 cosine schedule 的“冷却”阶段（cool down）反复训练，取损失的下包络（lower envelope）作为收敛损失的估计。这样可以部分缓解“何时算收敛”的不确定性。

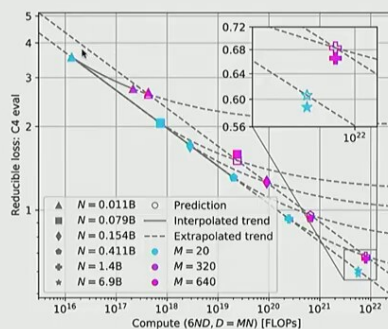
⁹视频画面时间区间：00:21:15 – 00:21:45。

2.7 Compute-Optimal 前沿

Side note – other ways of estimating chinchilla curves

Gadre et al propose other, curve-fitting based ways of doing similar things

Core idea – the ‘penalty’ from overtraining remains stable



$$L(N, D) = E + AN^{-\alpha} + BD^{-\beta}.$$

$$L(C, M) = E + (aM^{\alpha_C} + bM^{-\alpha_C}) C^{-\alpha_C},$$

where $\alpha_C = \alpha/2$, $a = A(1/6)^{-\alpha_C}$, $b = B(1/6)^{-\alpha_C}$ gives the relation to Equation (3). For a complete derivation, see Appendix B.

Stanford

Figure 10: Better scaling laws: Compute-Optimal 前沿——最优 (N^*, D^*) 随 C 的轨迹¹⁰

将所有 IsoFLOP 曲线的谷底连接起来，得到一条在 $(\log N, \log D)$ 平面上的直线。这条直线的斜率接近 1（即 $N^* \propto D^*$ ），截距给出 $D^*/N^* \approx 20$ 。

Tiny models with lots of data

The overall data-to-model ratio is very high (192), though they argue LLaMA architectures should have a higher ratio.

As for the concrete data-to-model ratio $\frac{D_{opt}}{N_{opt}}$, we notice that there is a huge gap in compute optimal regime of between ours and Hoffmann et al. (2022) despite that the trend of $\frac{D_{opt}}{N_{opt}}$ with compute C is aligned between ours and theirs. Specifically, the data size should be 192 times larger than the model size on average, as opposed to 20 times in Hoffmann et al. (2022). We note that this aligns with the observation in Section 4.3 and Figure 6.

Note that recent models like LLaMA 3 has *significantly* higher data-to-model ratios, suggesting that with more careful optimization, we might be able to go far beyond the 20*model_size rule of thumb.

Stanford

Figure 11: Better scaling laws: 完整 loss 拟合的残差与置信区间¹¹

讲师展示了拟合的残差图，强调拟合在中间计算量区间（ $10^{18} - 10^{21}$ FLOPs）非常准确，但在极小或极大区间外推时置信区间会变宽。这意味着：

¹⁰ 视频画面时间区间：00:25:00 – 00:25:30。

¹¹ 视频画面时间区间：00:27:30 – 00:28:00。

Scaling Law 的外推风险

从 10^{21} FLOPs 的实验拟合出的指数，外推到 10^{25} FLOPs（如 GPT-4 级别）时存在系统性误差风险。实际上，后来的研究（如 Llama 3、DeepSeek）发现，在极大计算量下，最优 D/N 比例可能进一步上升（“过度训练”趋势），这与 Chinchilla 的 $C^{0.5}$ 外推不完全一致。本节后半段将讨论这种偏离。

2.8 Chinchilla 的三种独立方法

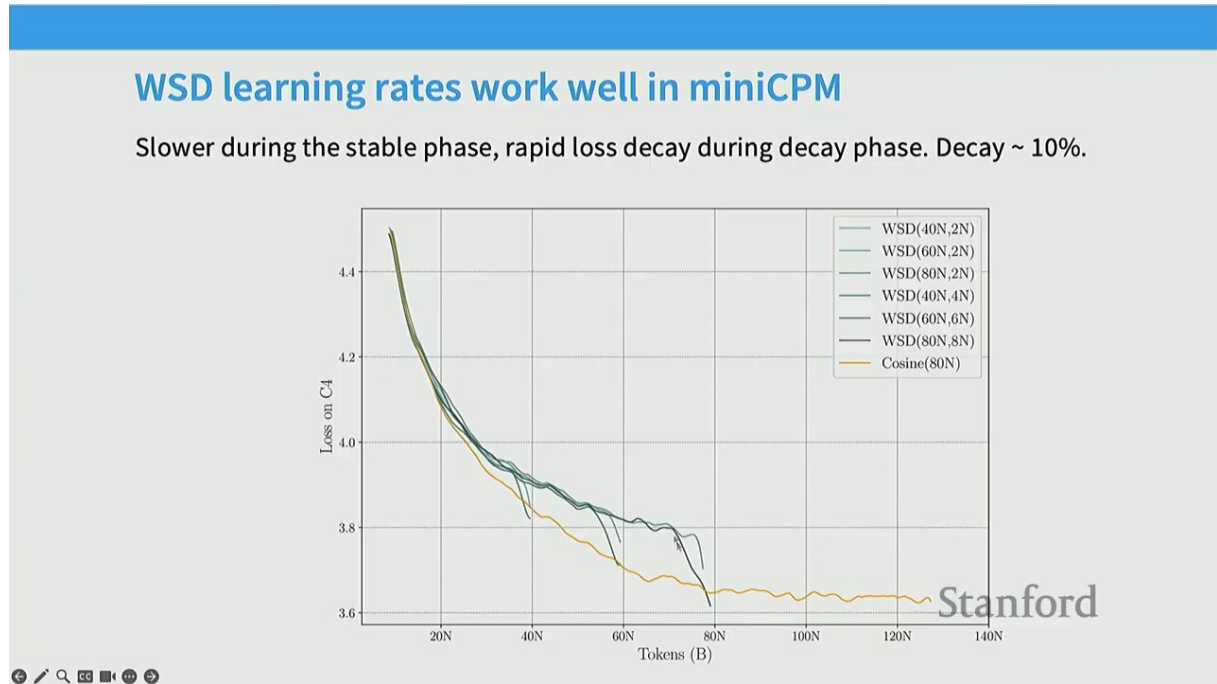


Figure 12: Better scaling laws: Chinchilla 三种独立方法交叉验证¹²

为增强结论的稳健性，Chinchilla 论文使用了三种方法论上独立的估计方法，交叉验证 $N^*(C)$ 和 $D^*(C)$ ：

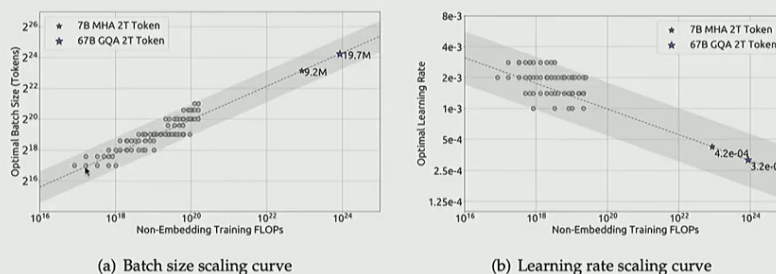
1. 方法一：参数化拟合（Approach 1）。直接拟合 $L(N, D) = E + A/N^{\alpha_N} + B/D^{\alpha_D}$ ，然后解析求最优。这是前面讲的标准流程。
2. 方法二：固定 N ，扫描 D （Approach 2 - IsoFLOP）。对每个固定的 N ，训练多个不同 D 的模型，找到使该 N 下损失最低的 D 。这给出 IsoFLOP 曲线族的谷底，不依赖参数化假设。
3. 方法三：固定计算量，扫描 (N, D) 组合（Approach 3）。对每个固定的 C ，训练多组 (N, D) （满足 $6ND = C$ ），直接找损失最低的组合。这是最直接的“实验测量”，无需任何参数化模型。

¹²视频画面时间区间：00:31:15 - 00:31:45。

Scaling analysis of learning rates

Small scale runs + collect 'near optimal' (within 0.25% of min) models.

$$\begin{aligned}\eta_{\text{opt}} &= 0.3118 \cdot C^{-0.1250} \\ B_{\text{opt}} &= 0.2920 \cdot C^{0.3271}\end{aligned}\quad (1)$$



Learning rate fit looks a bit questionable..

Stanford

Figure 13: Better scaling laws: 三种方法的 IsoFLOP 曲线对比与一致性检验¹³

三种方法的一致性

三种方法独立得到的 $N^*(C) \propto C^{0.5}$ 和 $D^*(C) \propto C^{0.5}$ 几乎完全一致。这种方法论交叉验证是 Chinchilla 结论可信的关键——单一方法的偏差被另外两种方法修正。讲师特别指出，方法二和方法三因为不依赖参数化假设，在计算量极大区间 ($> 10^{22}$ FLOPs) 的估计可能比方法一更可靠，因为参数化模型在那里的外推误差最大。

¹³ 视频画面时间区间：00:35:00 – 00:35:30。

3 Scaling Laws 的真正含义

3.1 它们在统计什么?

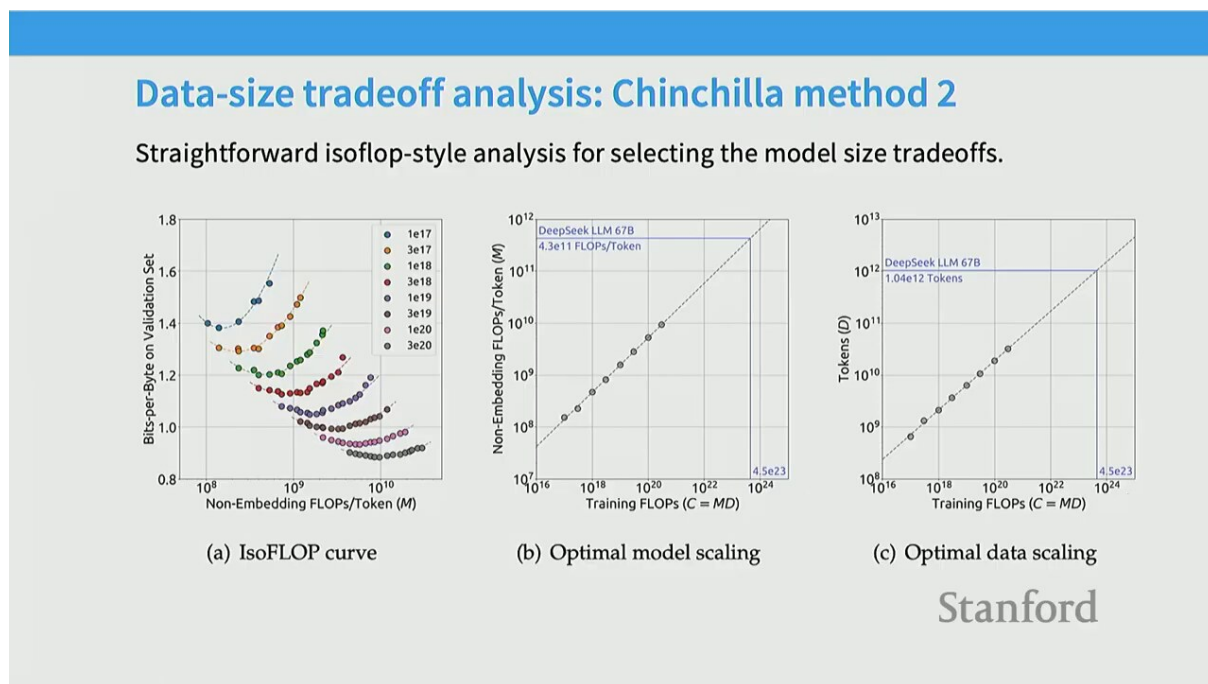


Figure 14: Scaling laws: what they mean——Scaling Law 是关于数据分布的统计陈述¹⁴

讲师在这一节提出了一个深刻的方法论问题：Scaling Law 到底在刻画什么？他的核心论点是：

Scaling Law 的本质

Scaling Laws 是关于特定数据分布的统计陈述。它们回答的问题是：“在分布 $\mathcal{D}$ 上，给定 (N, D) ，模型的期望损失是多少？”损失 L 是模型在分布 $\mathcal{D}$ 上的交叉熵，即：

$$L(N, D) = -\mathbb{E}_{x \sim \mathcal{D}} [\log p_{\theta}(x)]$$

- $p_{\theta}(x)$: 模型对样本 x 的预测概率
- $\mathcal{D}$: 目标数据分布（通常是训练数据所代表的互联网文本分布）
- L : 在该分布上的平均负对数似然

这意味着 Scaling Law 的一切结论——最优 N^* , D^* 、幂律指数 α ——都依赖于 $\mathcal{D}$ 。换一个数据分布（如代码 vs. 自然语言，或高质量过滤数据 vs. 原始 Common Crawl），拟合出的参数会完全不同。

¹⁴视频画面时间区间：00:37:30 – 00:38:00。

3.2 数据质量改变 Scaling Law

数据质量是最强的“调节旋钮”

如果用高质量、经过清洗和去重的数据训练，相同 (N, D) 下的损失会显著低于用原始网页爬取数据。这意味着：

- 高质量数据的 E （不可约损失）更低——因为数据“更可预测”。
- 高质量数据的 α 也可能更大——损失下降更快，因为每多一个 token 的边际信息量更高。
- 结果：最优 D^* 可能变小——你不需要那么多 token 就能达到相同损失。

这正是为什么 Llama 3 等模型即使训练 token 数远超 Chinchilla 最优值，性能仍在提升——它们用的是更高质量、更高效的数据。

3.3 下游任务 vs. 训练损失

Scaling Law 通常拟合的是训练损失（交叉熵）。但实际关心的往往是下游任务表现（如 MMLU、HumanEval）。两者之间存在复杂关系：

- 线性区：在大多数任务上，下游准确率与训练损失近似对数线性相关——损失每降低固定量，下游准确率提升固定比例。这就是著名的“log-linear scaling”。
- 相变区：某些任务（如 in-context learning、链式推理）存在涌现现象——损失降到某个阈值前，下游表现几乎为零；越过阈值后突然跃升。这种非线性使得从训练损失外推下游表现变得不可靠。

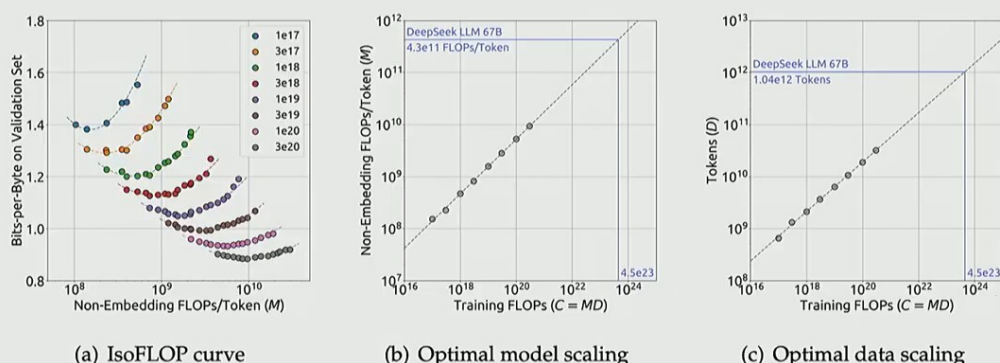
涌现能力的争议

Schaeffer et al. (2023) 指出，许多“涌现”可能只是评价指标的非线性（如 exact-match 从 0 跳到 1）造成的假象——改用连续指标（如 Brier score）后，相变变成平滑过渡。但无论如何，从训练损失预测下游表现需要谨慎，不能简单外推。

3.4 Scaling Law 的适用边界

Data-size tradeoff analysis: Chinchilla method 2

Straightforward isoflop-style analysis for selecting the model size tradeoffs.



Stanford

Figure 15: Scaling laws 的适用边界：何时外推会失效¹⁵

讲师总结了几种 Scaling Law 外推失效的典型情形：

1. 数据耗尽：Chinchilla 假设数据无限。但当 D^* 超过高质量数据的总量时（如互联网去重后约 10 – 20 万亿 token），无法继续按 $D^* \propto C^{0.5}$ 增长，模型被迫“数据受限”。
2. 架构变更：Scaling Law 是针对特定架构（如 dense Transformer）拟合的。换成 MoE、稀疏注意力、或全新架构，幂律指数会变。
3. 任务特异性：训练损失的外推对通用任务较准，但对特定能力（如数学推理、代码生成）的外推误差大。

4 超参数缩放：一个被忽视的问题

4.1 问题陈述：从 N_{small} 到 N_{large}

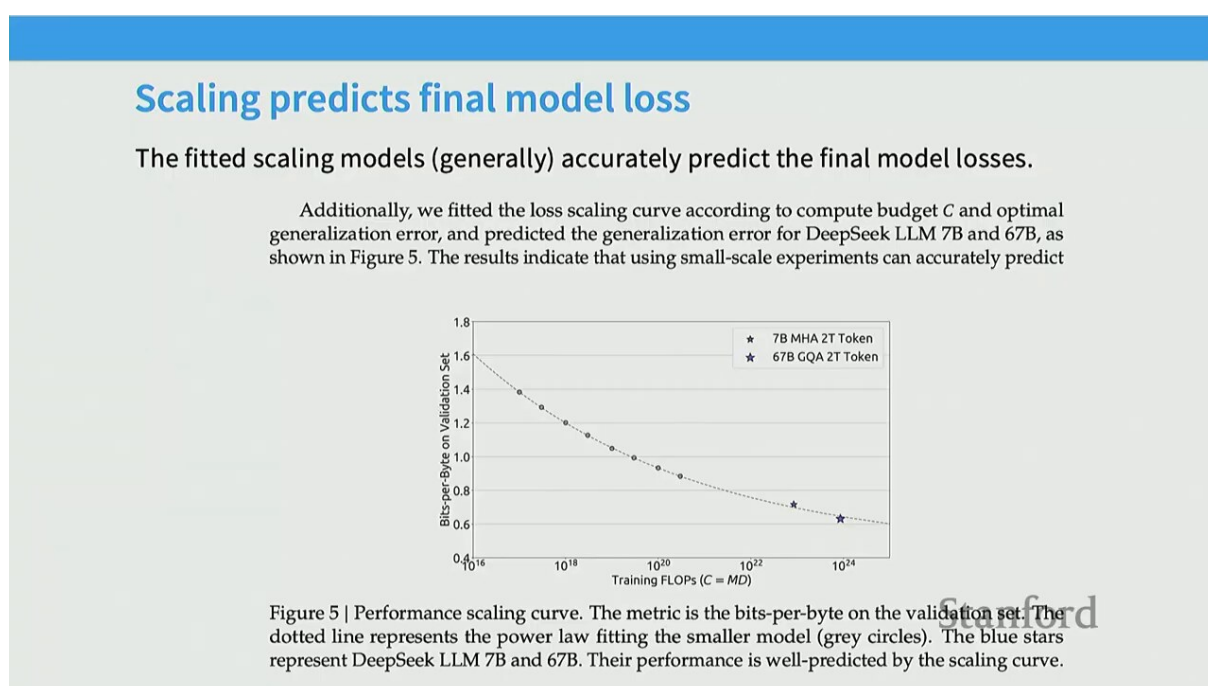


Figure 16: How should we scale learning rate and batch size?——超参数迁移的核心问题¹⁶

至此，Chinchilla 回答了“给定计算预算，最优 (N, D) 是多少”。但还有一个关键的工程问题没有回答：当模型从 N_{small} 扩大到 N_{large} 时，超参数应如何缩放？

具体而言，假设你在小模型上通过网格搜索找到了最优：

- 峰值学习率 η_{small}^*
- Batch size B_{small}^*
- 权重初始化的标准差 σ_{small}^*
- 各种正则化系数

现在你要训练一个 100 倍大的模型。这些超参数应该怎么变？直接复制显然不行——大模型用小模型的学习率会发散；用大 batch size 则梯度噪声结构不同。

¹⁵ 视频画面时间区间：00:39:00 – 00:39:30。

¹⁶ 视频画面时间区间：00:40:00 – 00:40:30。

为什么不能直接网格搜索？

在大模型上网格搜索学习率，每次试错都烧几十万美元。如果有一个理论框架能从小模型的超参数推导出大模型的超参数，就能省下巨额成本。这正是 μP (Maximal Update Parametrization) 要解决的问题。

4.2 宽度缩放：以宽度 m 为桥梁

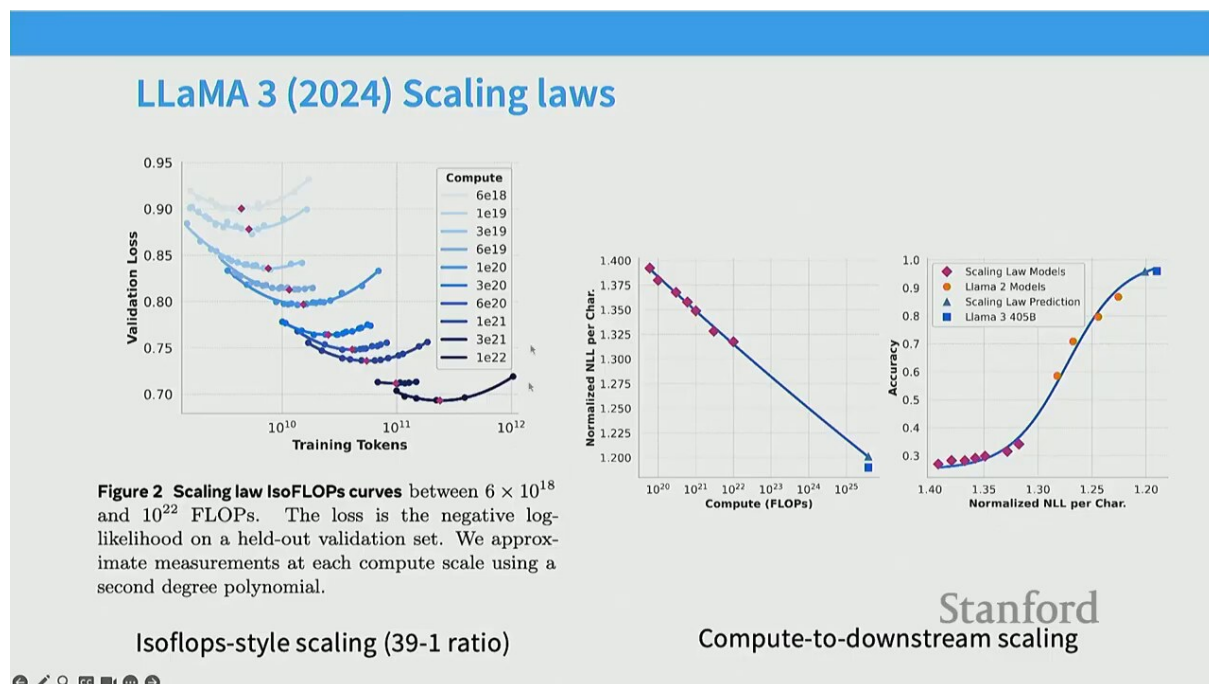


Figure 17: 超参数迁移：以网络宽度 m 为缩放桥梁¹⁷

讲师将问题简化为宽度缩放 (width scaling)：固定网络深度，把宽度（隐藏层维度）从 d 扩大到 $m \cdot d$ ，其中 m 是宽度缩放因子。分析激活值、梯度、权重更新如何随 m 变化。

¹⁷ 视频画面时间区间：00:42:30 – 00:43:00。

MiniMax-01 (2025)

Architecture scaling laws + Chinchilla method 1

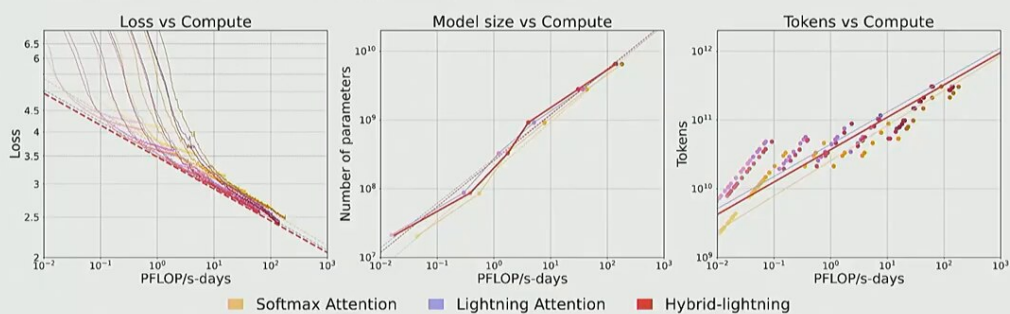


Figure 6 | **Summary of Scaling Laws.** Training curves (left) span models from 70M to 7B parameters. Optimal model size (center) and training tokens (right) are derived based on a specified compute budget estimation.

Stanford

Figure 18: Hyperparameter transfer: experimental setup——宽度扫描实验设计¹⁸

为什么要选宽度而非深度？因为：

- 宽度增加是最常见的扩缩方向（GPT 系列从 $d = 768$ 到 $d = 12288$ ）。
- 宽度缩放的数学分析比深度缩放更干净——每层的统计性质可以直接对 m 求期望。
- 宽度足够大时，网络的某些性质会“饱和”（收敛到无限宽极限），这正是 Neural Tangent Kernel 和 μP 的理论基础。

¹⁸视频画面时间区间：00:45:00 – 00:45:30。

4.3 实验观察：学习率应如何缩放

Recent scaling law recipes

CerebrasGPT

- Use muP to make hyperparams invariant to scale
- Directly use the chinchilla scaling formula

DeepSeek recipe

- Assume most transformer hypers are invariant to scale
- Do a scaling analysis on batch / LR to figure out optimal scaling
- IsoFLOP analysis to figure out model sizing
 - › Use a piecewise-linear schedule to make chinchilla scaling cheap.

miniCPM recipe

- Use muP to make transformer + LR invariant to scale
- Use a piecewise linear schedule to get sample for Chinchilla method 3 (curve fitting)

Recent (late 2024+) but less detailed

LLaMA 3 / Hunyuan

- Just isoflops (no other scaling details)

Minimax

- Architecture choice / decision scaling

Stanford

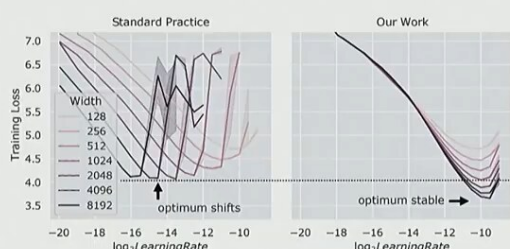
Figure 19: Hyperparameter transfer: 不同宽度下最优学习率的扫描结果¹⁹

讲师展示了实验结果：对每个宽度 m ，扫描学习率，找到使训练损失最低的 $\eta^*(m)$ 。关键观察是：

$$\eta^*(m) \propto \frac{1}{m}$$

Validating and understanding muP

“Scale invariant” hyperparameter tuning would be very useful



CerebrasGPT and miniCPT also use muP – is it actually useful?

Preprint

A Large-Scale Exploration of μ -Transfer

Lucas Dax Lingle

Stanford

Figure 20: Hyperparameter transfer: 学习率缩放公式 $\eta^* \propto 1/m$ 的实验验证²⁰

¹⁹ 视频画面时间区间：00:47:30 – 00:48:00。

²⁰ 视频画面时间区间：00:50:00 – 00:50:30。

即最优学习率与宽度成反比。宽度翻倍，学习率减半。这与朴素的“大模型用大学习率”直觉相反。

朴素直觉为何失效？

直觉上，参数越多，梯度方向越可靠，应该能用更大学习率。但实验显示相反。原因在于：宽度增加时，每个神经元接收的输入增多，梯度的方差也随之增大（因为更多独立信号的叠加）。如果不调小学习率，单步更新的“幅度”会爆炸，导致训练发散。 μP 的推导正是要精确量化这种方差增长。

4.4 验证：从小模型迁移到大模型

What is μP , anyway?

A Spectral Condition for Feature Learning

Greg Yang*
xAI

James B. Simon*
UC Berkeley & Imbue

Jeremy Bernstein*
MIT

(this is a very accessible ‘ μP for babies’ paper)

μP is based off the following assertion. As a function of the width of the network n_l ..

A1: The activations at initialization should remain $\Theta(1)$

A2: After one gradient step, the change in activation should be $\Theta(1)$

Stanford

Note: if individual activations are $\Theta(1)$, then the norm should be $\Theta(\sqrt{n_l})$

Figure 21: Hyperparameter transfer: 从小模型迁移到大模型的验证结果²¹

讲师展示了关键验证：在小模型（如 $d = 128$ ）上找到最优学习率 η_{small}^* ，然后按 $\eta_{\text{large}}^* = \eta_{\text{small}}^* / m$ 推算大模型（如 $d = 4096$, $m = 32$ ）的学习率。结果显示，这个推算值与大模型上独立网格搜索的最优值高度一致——迁移是有效的。

工程收益

μP 的迁移意味着：你只需要在小模型上做一次昂贵的超参数搜索，然后免费获得所有大模型的超参数。对于一个要训 10 个不同规模模型的团队，这是数量级的成本节省。这正是 μP 在工业界（如 Anthropic、Meta）被广泛采用的原因。

²¹ 视频画面时间区间：00:52:30 – 00:53:00。

5 Maximal Update Parametrization (μ P)

5.1 动机：从经验观察到理论框架

Deriving μ P (condition A1)

Suppose that we have a simple, deep linear network ($h_l = W_l h_{l-1}$) and we init $W_l \sim N(0, \sigma I_{n_l \times n_{l-1}})$ then by basic matrix concentration $\|W_l\|_* \rightarrow \sigma(\sqrt{n_{l-1}} + \sqrt{n_l})$ and,

$$\|h_l\|_2 \approx \|W_l\|_* \|h_{l-1}\|_2$$

Now let's pick $\sigma = \frac{\sqrt{n_l}}{\sqrt{n_{l-1}}} (\sqrt{n_l} + \sqrt{n_{l-1}})^{-1} = \Theta\left(\frac{1}{\sqrt{n_{l-1}}} \min\left(1, \sqrt{\frac{n_l}{n_{l-1}}}\right)\right)$. What happens?

Inductive assumption- $\|h_{l-1}\|_2 = \Theta(\sqrt{n_{l-1}})$

Inductive case - $\|W_l\|_* \rightarrow \sigma(\sqrt{n_{l-1}} + \sqrt{n_l}) = \frac{\sqrt{n_l}}{\sqrt{n_{l-1}}}$

$$\|h_l\|_2 = \sqrt{n_l} + o(\sqrt{n_l})$$

Stanford

[Comments – the $\approx$ here is a bit hand-wavy, as the min s.v. of a Gaussian is $\sqrt{n_l} - \sqrt{n_{l-1} - 1}$, and is non-uniform, like J.L.]

Figure 22: Maximal Update Parametrization (μ P): 理论框架的动机²²

前面的实验观察 $\eta^* \propto 1/m$ 是经验性的。Yang et al. (2021) 的 μ P 论文给出了一个原则性的理论框架，不仅能解释这个观察，还能推导出所有超参数（初始化方差、学习率、权重衰减等）的缩放律。

μ P 的核心思想：设计一种参数化方式，使得网络在任意宽度下，单步权重更新的“典型幅度”保持不变。“Maximal Update”之名由此而来——它最大化但不发散。

²² 视频画面时间区间：00:55:00 – 00:55:30。

5.2 激活值与梯度的尺度分析

Deriving muP (condition A1)

Suppose that we have a simple, deep linear network ($h_l = W_l h_{l-1}$) and we init $W_l \sim N(0, \sigma I_{n_l \times n_{l-1}})$ then by basic matrix concentration $\|W_l\|_* \rightarrow \sigma(\sqrt{n_{l-1}} + \sqrt{n_l})$ and,

$$\|h_l\|_2 \approx \|W_l\|_* \|h_{l-1}\|_2$$

Now let's pick $\sigma = \frac{\sqrt{n_l}}{\sqrt{n_{l-1}}} (\sqrt{n_l} + \sqrt{n_{l-1}})^{-1} = \Theta\left(\frac{1}{\sqrt{n_{l-1}}} \min\left(1, \sqrt{\frac{n_l}{n_{l-1}}}\right)\right)$. What happens?

Inductive assumption- $\|h_{l-1}\|_2 = \Theta(\sqrt{n_{l-1}})$

Inductive case - $\|W_l\|_* \rightarrow \sigma(\sqrt{n_{l-1}} + \sqrt{n_l}) = \frac{\sqrt{n_l}}{\sqrt{n_{l-1}}}$

$$\|h_l\|_2 = \sqrt{n_l} + o(\sqrt{n_l})$$

Stanford

[Comments – the $\approx$ here is a bit hand-wavy, as the min s.v. of a Gaussian is $\sqrt{n_l} - \sqrt{n_{l-1}} - 1$, and is non-uniform, like J.L.]

Figure 23: μP : 激活非消散条件——前向传播中激活的尺度分析²³

μP 的推导从分析前向传播中激活值的尺度开始。考虑一个宽度为 m 的全连接层：

$$h_j^{(l+1)} = \phi\left(\sum_{i=1}^m W_{ji}^{(l)} h_i^{(l)}\right)$$

- $h_j^{(l+1)}$: 第 $l+1$ 层第 j 个神经元的激活
- $W_{ji}^{(l)}$: 连接第 l 层第 i 个神经元到第 $l+1$ 层第 j 个神经元的权重
- ϕ : 激活函数（如 ReLU）
- m : 宽度（输入维度）

非消散条件

为使激活 $h_j^{(l+1)}$ 在任意宽度 m 下保持 $O(1)$ 的尺度（既不爆炸也不消失），权重初始化的标准差必须满足：

$$\text{std}(W^{(l)}) \propto \frac{1}{\sqrt{m}}$$

证明思路：假设 $h_i^{(l)}$ 和 $W_{ji}^{(l)}$ 独立且均值为零，则

$$\text{Var}\left(\sum_{i=1}^m W_{ji}^{(l)} h_i^{(l)}\right) = m \cdot \text{Var}(W) \cdot \text{Var}(h^{(l)})$$

要使这个方差与 m 无关（即 $O(1)$ ），需要 $\text{Var}(W) \propto 1/m$ ，即 $\text{std}(W) \propto 1/\sqrt{m}$ 。

这就是经典的 He 初始化（或 Xavier 初始化）的数学根源。它保证了前向传播中激活的稳定性。但 μP 还要保证反向传播中梯度的稳定性，以及权重更新的稳定性。

²³视频画面时间区间：00:57:30 – 00:58:00。

5.3 μP 的完整推导

Deriving μP (condition A2)

Now we need to deal with updates. Suppose we have the update ΔW_l on the weights. For SGD, on a linear layer, this looks like a rank-one loss-activation outer product.

$$\Delta W_l = -\eta_l \nabla_{h_l} \ell h_{l-1}^\top$$

Thus, $\|\Delta W_l h_{l-1}\|_2 = \|\Delta W_l\|_* \|h_{l-1}\|_2$. Now note that we have the update

$$\Delta h_l = W_l \Delta h_{l-1} + \Delta W_l (h_{l-1} + \Delta h_{l-1})$$

Assuming that the leading order terms don't cancel, we see that

- $W_l \Delta h_{l-1} = \Theta(\sqrt{n_l})$ from induction assumption + condition A1 argument
- $\Delta W_l h_{l-1} = \|\Delta W_l\|_* \sqrt{n_{l-1}}$ from above, thus $\|\Delta W_l\|_* = \Theta\left(\frac{\sqrt{n_l}}{\sqrt{n_{l-1}}}\right)$
- $\Delta W_l \Delta h_{l-1} = O(\|\Delta W_l\|_* \sqrt{n_{l-1}})$

Stanford

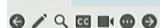


Figure 24: μP : 完整的数学推导——初始化、前向、反向、更新的联合尺度分析²⁴

μP 的完整推导需要同时约束三个尺度：前向激活、反向梯度、权重更新。讲师给出了推导的骨架。

μP 的三个尺度约束

设宽度为 m ，权重初始化标准差为 σ_w ，学习率为 η 。

约束 1（前向稳定）：激活 $h^{(l)}$ 不随 m 爆炸或消失：

$$\sigma_w \propto \frac{1}{\sqrt{m}}$$

约束 2（反向稳定）：梯度 $\partial L / \partial h^{(l)}$ 不随 m 爆炸：

类似地，需要反向传播中梯度的方差与 m 无关

约束 3（更新稳定）：权重更新 $\Delta W = -\eta \cdot \nabla W$ 的相对幅度 $|\Delta W|/|W|$ 不随 m 变化：

$$\frac{|\Delta W|}{|W|} = \frac{\eta \cdot |\nabla W|}{|W|} = O(1)$$

由于 $|W| \propto \sigma_w \propto 1/\sqrt{m}$ ，而 $|\nabla W|$ 在反向稳定下与 m 无关 ($O(1)$)，要使比值 $O(1)$ ，需要：

$$\eta \propto \frac{1}{\sqrt{m}} \cdot \frac{1}{O(1)} \cdot \sqrt{m} \Rightarrow \eta \propto \text{const or } 1/m$$

具体形式取决于层的类型（输入层、隐藏层、输出层）， μP 对不同层给出不同的缩放律。

²⁴视频画面时间区间：01:00:00 – 01:00:30。

5.4 初始化稳定性

Deriving muP (condition A2) part 2

Recall – we want all 3 terms of $\Delta h_l = W_l \Delta h_{l-1} + \Delta W_l (h_{l-1} + \Delta h_{l-1})$ to be $\Theta(\sqrt{n_l})$
And the key is to pick LR such that $\|\Delta W_l\|_* \sqrt{n_{l-1}} = \Theta(\sqrt{n_l})$. How can we do that?

Suppose that the loss update *also* scales $O(1)$. Then we can write down..

$$\Delta \ell = \Theta(\langle \Delta W_l, \nabla_{W_l} \ell \rangle) = \Theta(\|\Delta W_l\|_F \|\nabla_{W_l} \ell\|_F) = \Theta(\|\Delta W_l\|_* \|\nabla_{W_l} \ell\|_*)$$

Where we use the fact that $\Delta W_l = -\eta \nabla_{W_l} \ell$ in standard SGD updates.

Now plug in $\Delta \ell = O(1)$, $\|\Delta W_l\|_* = \Theta\left(\frac{\sqrt{n_l}}{\sqrt{n_{l-1}}}\right)$ to get that $\|\nabla_{W_l} \ell\|_* = \Theta\left(\frac{\sqrt{n_{l-1}}}{\sqrt{n_l}}\right)$

Finally, from the previous slide, recall that $\Delta W_l = -\eta_l \nabla_{h_l} \ell h_{l-1}^T$ and thus

$$\eta_l = \Theta\left(\frac{n_l}{n_{l-1}}\right)$$

Stanford

[with Adam, $\|\Delta W_l\|_* \sqrt{n_{l-1}} = \Theta(1)$]

Figure 25: μP : 初始化稳定性——不同宽度下的初始激活分布²⁵

讲师展示了实验验证：在 μP 参数化下，不同宽度的网络在初始化时的激活分布几乎重合（尺度都是 $O(1)$ ）。而在标准的 NTP (Standard Parametrization) 下，宽网络的初始激活会发散或消失。

参数化	权重初始化	学习率
Standard (SP/NTP)	$\sigma_w \propto 1/\sqrt{m}$	$\eta \propto 1$ (常数)
μP	$\sigma_w \propto 1/\sqrt{m}$ (隐藏层)	$\eta \propto 1/m$ (输出层)
	$\sigma_w \propto 1/m$ (输出层)	$\eta \propto 1$ (隐藏层)
NTK 参数化	$\sigma_w \propto 1$	$\eta \propto 1/m$

三种参数化的谱系

NTK 参数化 (Jacot et al., 2018)：无限宽极限下，网络训练等价于核回归。学习率 $\eta \propto 1/m$ ，使得无限宽时训练仍然有意义（否则更新太小，永远学不动）。

Standard 参数化 (SP)：深度学习标准做法。初始化用 He，学习率为常数。问题：宽度变化时，更新的相对幅度会变化，超参数无法迁移。

μP ：介于两者之间。通过精心设计不同层的初始化和学习率缩放，使得有限宽度下更新幅度恒定，从而支持从小到大的超参数迁移。 μP 在 $m \rightarrow \infty$ 时收敛到 NTK 极限，但在有限宽度下比 SP 更稳定。

²⁵ 视频画面时间区间：01:02:30 – 01:03:00。

5.5 宽度缩放推导：输出层 vs. 隐藏层

Implementation in Cerebras GPT		
We now have the background to follow cerebrasGPT		
Standard Parameterization (SP)		Maximal Update (μP)
Empirically Tuned Values		
$d_{\text{model, base}}$	—	256
η_{base}	Must tune for each model size	6e-3
σ_{base}	0.02	0.08
m_{emb}	—	10.0
Formulas		
Variables	Standard Parameterization (SP)	Maximal Update (μP)
W	A multiplicative or fully-connected weights tensor	$W_{\text{emb}} \sim N_{\text{trunc}}(0, \sigma_{\text{base}}^2)$
b	A bias weights tensor	$\eta_{\text{emb}} = \eta_{\text{base}}$
X, Y	Activation tensors: layer input, output, respectively	$Y_{\text{emb}} = \text{embed}(X)$
$d_{\text{model, base}}$	Proxy (base) model's layer width	$Y_{\text{emb}} = m_{\text{emb}} \cdot \text{embed}(X)$
d_{model}	Width of each layer	$W_{\gamma} \sim 1, b_{\beta} \sim 0$
d_{head}	Size of each attention head	$\eta_{\text{LN}} = \eta_{\text{base}}$
η_{base}	Combined token + position embedding function	$b \sim 0$
σ_{base}	The base learning rate (LR): Maximum in training schedule	$\eta_b = \eta_{\text{base}}$
m_{width}	The base initialization standard deviation	$\text{softmax}\left(\frac{Q^T K}{d_{\text{head}}}\right) V$
m_{emb}	Layer width multiplier: $d_{\text{model}}/d_{\text{model, base}}$	$W_{\text{qkv}} \sim N_{\text{trunc}}(0, \sigma_{\text{base}}^2)$
	Embedding output multiplier	$W_{\text{qkv}} \sim N_{\text{trunc}}(0, \sigma_{\text{base}}^2/m_{\text{width}})$
		$\eta_{\text{qkv}} = \eta_{\text{base}}$
		$W_o \sim N_{\text{trunc}}(0, \frac{\sigma_{\text{base}}^2}{2 \cdot m_{\text{width}} \cdot m_{\text{layers}}})$
		$\eta_o = \eta_{\text{base}}/m_{\text{width}}$
		$W_{\text{fn1}} \sim N_{\text{trunc}}(0, \sigma_{\text{base}}^2)$
		$W_{\text{fn1}} \sim N_{\text{trunc}}(0, \sigma_{\text{base}}^2/m_{\text{width}})$
		$\eta_{\text{fn1}} = \eta_{\text{base}}/m_{\text{width}}$
		$W_{\text{fn2}} \sim N_{\text{trunc}}(0, \frac{\sigma_{\text{base}}^2}{2 \cdot m_{\text{width}} \cdot m_{\text{layers}}})$
		$W_{\text{fn2}} \sim N_{\text{trunc}}(0, \frac{\sigma_{\text{base}}^2}{m_{\text{width}}})$
		$\eta_{\text{fn2}} = \eta_{\text{base}}/m_{\text{width}}$
		$Y_{\text{logits}} = W_{\text{unemb}} X$

Figure 26: μP：宽度缩放推导——输出层与隐藏层的差异化处理²⁶

μP 的精妙之处在于它对不同层采用不同的缩放律。讲师详细推导了为什么：

输出层的特殊处理

输出层（读出层，readout layer）连接最后一个隐藏层到输出。它的权重 $W_{\text{out}} \in \mathbb{R}^{1 \times m}$ （假设单输出）。

前向：输出 $y = \sum_{i=1}^m W_{\text{out}, i} \cdot h_i$ 。要使 y 的尺度 $O(1)$ ，需要 $\text{Var}(W_{\text{out}}) \propto 1/m$ ，即 $\sigma_{\text{out}} \propto 1/\sqrt{m}$ （与隐藏层相同）。

但反向梯度不同： $\partial L / \partial W_{\text{out}, i} = (\partial L / \partial y) \cdot h_i$ 。由于 h_i 是 $O(1)$ ，梯度也是 $O(1)$ 。

更新的相对幅度：

$$\frac{|\Delta W_{\text{out}}|}{|W_{\text{out}}|} = \frac{\eta \cdot O(1)}{O(1/\sqrt{m})} = \eta \cdot O(\sqrt{m})$$

要使这个比值 $O(1)$ （不随 m 爆炸），需要 $\eta_{\text{out}} \propto 1/\sqrt{m}$ ，或者更激进地，将输出层的初始化设为 $\sigma_{\text{out}} \propto 1/m$ （而非 $1/\sqrt{m}$ ），这样：

$$\frac{|\Delta W_{\text{out}}|}{|W_{\text{out}}|} = \frac{\eta \cdot O(1)}{O(1/m)} = \eta \cdot O(m) \Rightarrow \eta_{\text{out}} \propto 1/m$$

μP 选择后者（ $\sigma_{\text{out}} \propto 1/m$ ， $\eta_{\text{out}} \propto 1/m$ ），使得输出层在宽度变化时行为最稳定。

²⁶ 视频画面时间区间：01:05:00 – 01:05:30。

5.6 学习率与宽度的最终关系

Deriving muP (condition A2) part 2

Recall – we want all 3 terms of $\Delta h_l = W_l \Delta h_{l-1} + \Delta W_l (h_{l-1} + \Delta h_{l-1})$ to be $\Theta(\sqrt{n_l})$
And the key is to pick LR such that $\|\Delta W_l\|_* \sqrt{n_{l-1}} = \Theta(\sqrt{n_l})$. How can we do that?

Suppose that the loss update also scales $O(1)$. Then we can write down..

$$\Delta \ell = \Theta(\langle \Delta W_l, \nabla_{W_l} \ell \rangle) = \Theta(\|\Delta W_l\|_F \|\nabla_{W_l} \ell\|_F) = \Theta(\|\Delta W_l\|_* \|\nabla_{W_l} \ell\|_*)$$

Where we use the fact that $\Delta W_l = -\eta \nabla_{W_l} \ell$ in standard SGD updates.

Now plug in $\Delta \ell = O(1)$, $\|\Delta W_l\|_* = \Theta\left(\frac{\sqrt{n_l}}{\sqrt{n_{l-1}}}\right)$ to get that $\|\nabla_{W_l} \ell\|_* = \Theta\left(\frac{\sqrt{n_{l-1}}}{\sqrt{n_l}}\right)$

Finally, from the previous slide, recall that $\Delta W_l = -\eta_l \nabla_{h_l} \ell h_{l-1}^T$ and thus

$$\eta_l = \Theta\left(\frac{n_l}{n_{l-1}}\right)$$

Stanford

[with Adam, $\|\Delta W_l\|_* \sqrt{n_{l-1}} = \Theta(1)$]

Figure 27: μP : 学习率 $1/m$ 关系的最终推导与验证²⁷

综合所有层的分析， μP 给出的超参数缩放律为：

层类型	权重初始化 σ_w	学习率 η	特征函数
输入层 (embedding)	1 (固定)	1 (固定)	无缩放
隐藏层	$1/\sqrt{m}$	1 (常数)	He 初始化
输出层 (readout)	$1/m$	$1/m$	关键差异点

μP 的实践口诀

核心差异在输出层： μP 把输出层的权重初始化设为 $1/m$ （比标准的 $1/\sqrt{m}$ 更小），学习率也设为 $1/m$ 。这样，无论网络多宽，输出层的单步更新相对幅度都保持恒定。

迁移规则：在小模型（宽度 m_{small} ）上搜索最优学习率 η_{small}^* ，然后大模型（宽度 m_{large} ）的学习率为：

$$\eta_{\text{large}}^* = \eta_{\text{small}}^* \cdot \frac{m_{\text{small}}}{m_{\text{large}}}$$

由于隐藏层学习率不随 m 变（常数），只有输出层需要这个缩放。实践中，框架（如 PyTorch 的 μP 实现）会自动处理这种差异化缩放。

5.7 Batch size 的缩放

μP 主要解决了学习率和初始化的迁移。Batch size 是另一个需要缩放的超参数。讲师简要提及了相关的经验规则：

- 临界 batch size：存在一个 B_{crit} ，当 batch size 低于它时，训练效率（每步进度）与 batch size 成正比；高于它时，边际收益递减。
- B_{crit} 随训练进度增长：训练后期，梯度方差减小，可以用更大 batch size。
- 缩放规则： $B_{\text{crit}}(C) \propto C$ ，即大计算量允许更大 batch size。

²⁷ 视频画面时间区间：01:07:30 – 01:08:00。

Batch size 缩放与 Chinchilla 的关系

Chinchilla 的训练 token 数 D 与 batch size B 和训练步数 S 的关系是 $D = B \cdot S$ 。如果 batch size 太小，需要更多步数（更慢）；如果太大，每步的梯度噪声不足，收敛质量下降。最优 batch size 通常取 B_{crit} 的 2-4 倍，在效率和质量间取得平衡。

6 μP 的总结与实践意义

6.1 Summary: μP 提供了什么

Deeper dive into μP

Recall – μP is a scaling procedure for hyperparams (as a function of width)

Param	Init Variance ($\odot$)	Adam LR ($\odot$)	Init Variance (Exact)	Adam LR (Exact)
$\mathbf{W}^E$	1	1	1	α
$\mathbf{W}^{AQ}$	$1/M$	$1/M$	$1/M$	$\alpha P/M$
$\mathbf{W}^{AK}$	$1/M$	$1/M$	$1/M$	$\alpha P/M$
$\mathbf{W}^{AV}$	$1/M$	$1/M$	$1/M$	$\alpha P/M$
$\mathbf{W}^{AO}$	$1/(HD)$	$1/(HD)$	$1/M$	$\alpha P/M$
$\mathbf{W}^{FI}$	$1/M$	$1/M$	$1/M$	$\alpha P/M$
$\mathbf{W}^{FO}$	$1/F$	$1/F$	$0.25/M$	$\alpha P/M$
$\mathbf{W}^U$	$1/M^2$	$1/M$	$1/M^2$	$\alpha P/M$

Preprint

A Large-Scale Exploration of μ -Transfer

Lucas Dax Lingle

Table 2: μP scaling rules for transformers; a rule for attention scale is detailed in text below.

In addition, μP uses an attention scale of $\tau^{-1} = \odot(1/D)$ instead of the usual $\tau^{-1} = 1/\sqrt{D}$. For simplicity, we use $\tau^{-1} = 1/D$, since in preliminary experiments we observed only a small improvement from using smaller multiples of $1/D$. Note that for D fixed across model widths M , any constant $\tau^{-1} \neq 0$ technically complies with μP (Yang et al., 2021) but in the experiments, τ^{-1} will be shown to have a major impact on performance and transfer.

Embedding

Attention params

Input/output MLP MM

Softmax linear

Figure 28: Summary: μP 提供了原则性的超参数迁移框架²⁸

μP 的三大贡献

1. 理论统一: μP 统一了 Standard Parametrization (有限宽度主导) 和 NTK Parametrization (无限宽度极限), 证明两者是连续谱的两个端点。
2. 实践迁移: μP 使得在小模型上搜索的超参数可以直接迁移到大模型, 省去昂贵的重复搜索。这是其被工业界广泛采用的主因。
3. 可解释性: μP 揭示了“为什么大模型需要小学习率”——不是经验规律, 而是激活和梯度方差随宽度增长的必然结果。

²⁸ 视频画面时间区间: 01:10:00 – 01:10:30。

6.2 μ P 与 Chinchilla 的关系

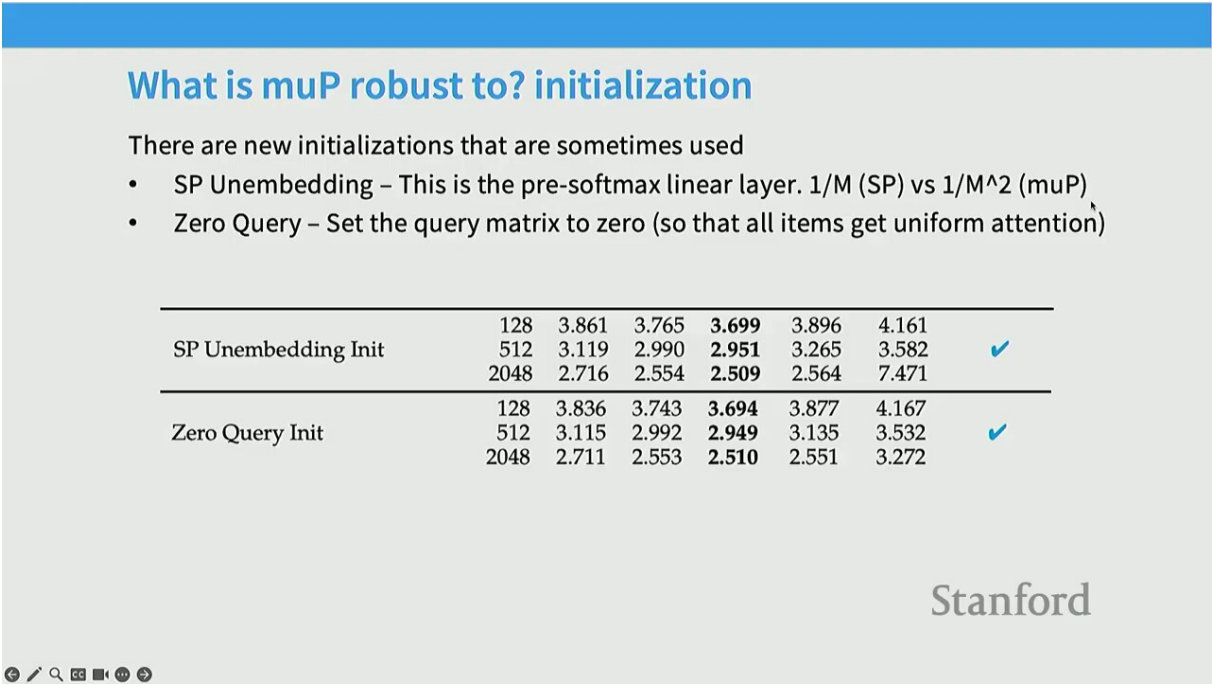


Figure 29: 总结与展望： μ P 与 Chinchilla 的互补关系²⁹

μ P 和 Chinchilla 回答的是两个不同但互补的问题：

维度	Chinchilla	μ P
回答的问题	给定计算 C ，最优 (N^*, D^*) ?	给定宽度 m ，最优超参数?
关注层面	宏观资源分配	微观训练动态
依赖的实验	多组 (N, D) 训练到收敛	多组宽度扫描学习率
产出	幂律指数 α ，最优比 D^*/N^*	缩放律 $\eta \propto 1/m$, $\sigma \propto 1/m$
外推风险	极大计算量下偏差（数据耗尽）	深度变化时需修正（ μ P 假设宽度主导）

两者结合，构成了“如何训练大模型”的完整工程指南：Chinchilla 告诉你“训多大、训多少 token”， μ P 告诉你“用什么学习率、怎么初始化”。

6.3 实际模型如何偏离 Chinchilla

过度训练（Over-training）趋势

现代大模型普遍偏离 Chinchilla 的 $D^*/N^* \approx 20$ 建议，采用更大的 D/N 比：

- Llama 3 8B：80 亿参数，训练 15 万亿 token， $D/N \approx 1875$ ——是 Chinchilla 建议的近百倍。
- DeepSeek-V2：2360 亿参数（MoE），训练 8.1 万亿 token。
- Qwen2.5：0.5B 到 72B 全系，普遍过度训练。

为什么？因为 Chinchilla 只考虑训练计算最优，而实际部署还要考虑推理成本。一个小但训练充分的模型，推理更便宜。这种“inference-aware scaling”使得最优 D/N 远大于 20。

²⁹ 视频画面时间区间：01:15:00 – 01:15:30。

Inference-Aware Scaling 的逻辑

设总成本 = 训练成本 + 推理成本。训练成本 $C_{\text{train}} \propto N \cdot D$ ，推理成本 $C_{\text{inference}} \propto N \cdot Q$ (Q 是推理查询次数)。

若 Q 很大 (如面向数亿用户的 API)，推理成本主导，应选更小的 N 和更大的 D (过度训练)，使相同性能下推理更便宜。

形式化地，最优条件变为：

$$\min_{N,D} [6ND + \lambda \cdot N \cdot Q] \quad \text{s.t.} \quad L(N, D) \leq L_{\text{target}}$$

其中 λ 是推理的单位成本系数。解出的 D^*/N^* 会随 $\lambda \cdot Q$ 增大而上升。

6.4 μP 的局限与开放问题

μP 没有解决的问题

1. 深度缩放: μP 主要分析宽度。深度增加时，梯度消失/爆炸需要额外处理 (如残差连接、LayerNorm)， μP 的缩放律需要修正。
2. 非全连接结构: Attention 层、MoE 的专家路由、稀疏结构等， μP 的标准推导不完全适用，需要针对性扩展。
3. 优化器影响: μP 分析的是 SGD 风格的更新。Adam 等自适应优化器的缩放律更复杂，因为它们对每个参数有独立的二阶矩估计。
4. 数据依赖: μP 假设数据分布固定。换数据集，最优超参数可能不同， μP 的迁移需要重新校准。

7 总结与延伸

7.1 本节核心论点

五个核心论点

1. Chinchilla 比 Kaplan 更可信: 通过参数化损失 $L(N, D) = E + A/N^{\alpha_N} + B/D^{\alpha_D}$ 和三种独立方法交叉验证，得出 $N^*, D^* \propto C^{0.5}$ ，最优 $D^*/N^* \approx 20$ 。GPT-3 据此被证明严重欠训练。
2. Scaling Law 是数据依赖的统计陈述: 换数据分布，幂律指数和最优分配都会变。数据质量是最强的调节旋钮——高质量数据使最优 D^* 减小。
3. 超参数迁移是独立的工程问题: Chinchilla 不回答“学习率怎么调”。 μP 填补了这个空白，从小模型推导大模型超参数，省下巨额搜索成本。
4. μP 的核心是输出层缩放: $\sigma_{\text{out}} \propto 1/m$, $\eta_{\text{out}} \propto 1/m$ ，使任意宽度下更新幅度恒定。这与 Standard Parametrization 的关键差异在输出层。
5. 实际模型偏离 Chinchilla 是理性的: Inference-aware scaling 使最优 D/N 远大于 20，因为推理成本主导，倾向于“小模型过度训练”。

7.2 下节预告

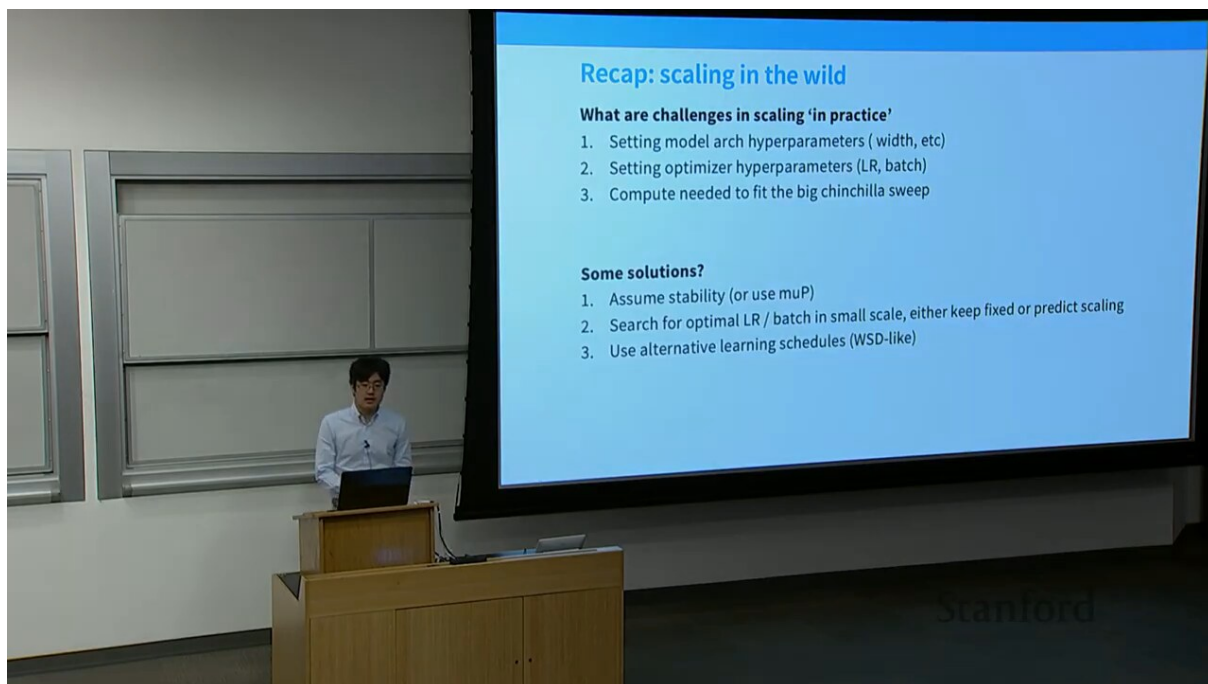


Figure 30: 课程结语：Scaling Laws 之后的下一步³⁰

Scaling Laws 两连讲结束。后续课程将转向数据工程（数据过滤、去重、混合比例）和对齐（RLHF、DPO），这些都是在 Scaling Law 框架内提升“算法效率”的具体手段。讲师最后强调：Scaling Laws 不是终点，而是理性分配资源的起点——知道每个 token、每个参数的边际收益，才能做出不浪费的工程决策。

7.3 配套阅读

论文	对应知识点
Kaplan et al. (2020) Scaling Laws for Neural LMs	Kaplan 式经验 Scaling Law
Hoffmann et al. (2022) Training Compute-Optimal LLMs (Chinchilla)	Chinchilla 方法论
Yang et al. (2021) Tensor Programs V: Tuning Large Neural Networks via μP	μP 理论
Yang & Hu (2021) Feature Learning in Infinite-Width Networks	μP 与 NTK 的关系
Hoffmann et al. – An Empirical Analysis of Compute-Optimal LLM	三种方法交叉验证
Schaeffer et al. (2023) Are Emergent Abilities of LLMs a Mirage?	涌现能力的争议

³⁰ 视频画面时间区间：01:18:15 – 01:18:45。